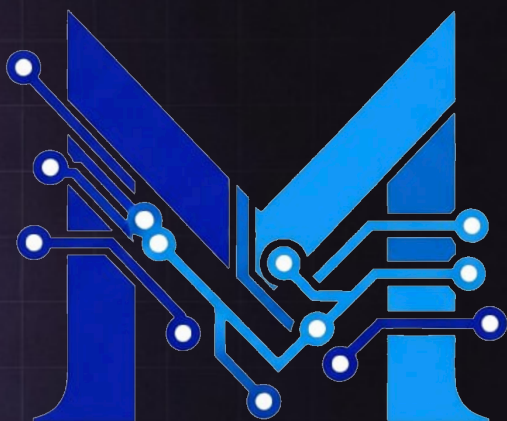


Snuggle

Security Review



1 April, 2026

Prepared by: Valves Security

Lead Auditors: Vesko210 | Merulez99

Contents

1	About Valves Security	3
2	Disclaimer	3
3	Risk Classification	3
4	Protocol Summary	4
5	Executive Summary	4
6	Findings	5
	Medium Findings	5
	[M-01] Pool removal is permanently blockable by any user via dust positions	5
	[M-02] Reward Adapter Rotation Causes Withdraw DoS for All Currently-Staked Positions	5
	Low/Info Findings	6
	[L-01] Cross-DEX TokenId collision overwrites position in base variants	6
	[L-02] calculateCenteredRange Mints One-Sided, Off-Center Positions When rangeWidthBps <= tickSpacing	7
	[L-03] Retroactive performance fee application charges existing positions at new rate on already-accrued earnings	9
	[L-04] renounceOwnership not blocked in SnuggleVaultUpgradeable	10
	[L-05] Missing reward adapter validation in updatePoolRewardAdapter	10
	[L-06] Adding a pool with a new token breaks fee harvesting until the admin re-registers the helper	11
	[L-07] Tick boundary clamping can produce inverted ranges at extreme ticks	12
	[I-01] clearReferrer leaves stale data in ReferralTracker when reassigning to a different referrer	13

1 About Valves Security

Valves Security is a Web3 security team founded by Veselin and Valeri. With deep experience across EVM-based protocols, Cosmos (Go) chains, and native Rust systems, we help projects identify and eliminate critical vulnerabilities. With a strong background and a strong team of freelancers we can assure a top quality audit.

Book a Security Review at valvesecurity.com or message us on X Valves Security

2 Disclaimer

The Valves team makes all effort to find as many vulnerabilities in the code in the given time period. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the implementation. We stand by our process, yet encourage follow-up audits and meaningful bug bounty incentives for continued whitehat protection.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- High - leads to a significant material loss of assets in the protocol or a group of users
- Medium - leads to a moderate material loss of assets in the protocol, moderately harms a group of users or to some disruption of the protocol's functionality
- Low - No funds at risk

Likelihood

- High - attack path is possible with reasonable assumptions. The issue is almost certain to happen.
- Medium - only a conditionally incentivized attack vector, but still relatively likely

- Low - has unlikely assumptions or requires a significant stake by the attacker

4 Protocol Summary

Snuggle is an automated yield platform that helps you earn trading fees from your crypto. Deposit Bitcoin, ETH, or stablecoins, and Snuggle's smart rebalancing technology optimizes your position to maximize fee earnings while minimizing losses.

This audit covers the smart contracts used by both **Snuggle** and **MaxFi**.

5 Executive Summary

Overview

Project	Snuggle
Commit Hash	-
Timeline	1/04/2026 – 14/04/2026

Scope

Id	Files in scope
1	src/*

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	2
Low/Info Risk	8

6 Findings

Medium Findings

[M-01] Pool removal is permanently blockable by any user via dust positions

Fixed

Impact: Low

Likelihood: High

Description

`removePool` iterates every active position in the vault and reverts if any belongs to the target pool:

```
1 function removePool(bytes32 poolId) external onlyVaultOwner {
2     uint256 count = vault.totalActivePositions();
3     for (uint256 i = 0; i < count; i++) {
4         uint256 tokenId = vault.allPositionIds(i);
5         (,bytes32 posPoolId,,,,,,,,,,,,,) = vault.positions(tokenId);
6         if (posPoolId == poolId) revert PoolHasActivePositions();
7     }
8     vault.removePool(poolId);
9 }
```

Any user can permanently prevent pool removal through two vectors: Dust position griefing: A user opens a minimal liquidity position in the target pool and never withdraws. The position stays active indefinitely, and the owner has no mechanism to force-close it. The vault owner loses the ability to deprecate pools.

Recommendation

Forgotten dust positions cannot be closed by the owner. Either allow pool removal while active positions exist, or add an admin function that can force-close positions while the vault is paused.

Team Response

Fixed: Fix (remove redundant guard) in codebase for future; live deployments need no upgrade — guard is non-custodial, positions withdraw fine after deactivation

[M-02] Reward Adapter Rotation Causes Withdraw DoS for All Currently-Staked Positions

Acknowledged

Impact: Medium

Likelihood: Low

Description

`updatePoolRewardAdapter()` replaces the pool's reward adapter reference without migrating NFTs that are currently staked through the old adapter. The new adapter has no record of those positions, so the vault's staked state and its physical NFT custody become desynchronized across the rotation boundary.

```
1 // SnuggleVaultUpgradeable.sol:L967-L979
2 function updatePoolRewardAdapter(bytes32 poolId, address newRewardAdapter) external
  onlyAdminSatellite {
3   PoolConfig storage config = approvedPools[poolId];
4   if (!config.active) revert PoolNotApproved();
5
6   if (newRewardAdapter != address(0)) {
7     address nftManager = config.positionAdapter.positionManager();
8     INonfungiblePositionManager(nftManager).setApprovalForAll(newRewardAdapter, true);
9     emit AdapterApproved(newRewardAdapter, nftManager);
10  }
11
12  config.rewardAdapter = IRewardAdapter(newRewardAdapter);
13  emit PoolRewardAdapterUpdated(poolId, newRewardAdapter);
14 }
```

The withdraw path reads `poolConfig.rewardAdapter` from storage at call time and routes through `StakingManager.tryUnstake`:

```
1 // StakingManager.sol:L188-L190
2 if (address(rewardAdapter) == address(0)) return (0, 0, 0);
3 if (!rewardAdapter.isStaked(tokenId)) return (0, 0, 0);
```

After rotation, the new adapter returns `isStaked(tokenId) == false` for every previously-staked position, so `tryUnstake` silently returns zero. The vault then calls `adapter.collect(tokenId, address(this))` and `adapter.decreaseLiquidity(tokenId, ...)` on the position manager (SnuggleVaultUpgradeable.sol:L553,L583), which revert because the NFT is physically held by the old adapter's gauge contract, not by the vault. Harvest and staked-reward claim paths fail for the same reason.

Every LP position that is currently staked at the moment of rotation experiences a withdraw DoS until recovery is performed. Principal and accrued rewards are not lost as long as recovery is possible.

`updatePoolRewardAdapter` places no restriction on the target address, so the admin can re-rotate back to the previous adapter in a subsequent transaction.

Recommendation

Require the pool to have no outstanding staked positions before allowing `updatePoolRewardAdapter` to over-write `config.rewardAdapter`, or perform an atomic unstake-and-restake of existing positions within the rotation flow.

Team Response

Acknowledged: Won't-fix — operational invariant (adapters never rotated on live pools; new strategy = new deployment)

Low/Info Findings

[L-01] Cross-DEX tokenId collision overwrites position in base variants

Fixed

The vault's internal positions mapping is keyed by raw NFT tokenId with no namespace for the originating position manager contract. The Base-chain variants support multiple DEX adapters simultaneously (Uniswap V3, PancakeSwap V3, Aerodrome Slipstream), each of which maintains its own independent NonfungiblePositionManager contract with an independently incrementing tokenId counter starting from 1. When two different DEX pools each

mint their N-th position, they produce identical tokenIds. The `_createPosition` function writes to `positions[tokenId]` unconditionally, with no guard to detect that a record already exists for that ID from a different manager:

```
1 // snuggle-base/SnuggleVaultUpgradeable.sol:L1077
2 positions[tokenId] = UserPosition({
3   tokenId: tokenId,
4   owner: msg.sender,
5   poolId: poolId,
6   // ...
7 });
```

When User2 deposits into a DEX pool that produces the same tokenId as an existing position owned by User1, `positions[tokenId]` is silently overwritten with User2's data. User1's position record — including owner address, pool ID, and tick range — is permanently destroyed without any revert or event. Secondary effects include duplicate entries in `allPositionIds` and `userPositions[user2]`, corrupting the $O(1)$ removal invariant that relies on position indices being unique.

- User1's `withdraw(tokenId)` permanently reverts with `NotPositionOwner` because the overwritten record now points to User2 as owner.
- User1 loses total access to their deposited LP liquidity with no recovery path — the admin's `rescueERC721` cannot help because the NFT is still in the vault under the correct DEX manager, but the position record no longer recognizes User1 as owner.
- `allPositionIds` accumulates a duplicate entry, inflating `totalActivePositions` by 1 and potentially breaking downstream gas calculations that assume unique IDs.
- The collision is silent: no event, no revert. Neither user nor admin receives any on-chain signal.

Recommendation

Add a one-line existence guard at the top of `_createPosition` before the unconditional write:

```
1 function _createPosition(uint256 tokenId, bytes32 poolId, ...) internal {
2   if (userPositions[msg.sender].length >= maxPositionsPerUser) revert
3     MaxPositionsPerUserReached();
4   if (allPositionIds.length >= maxTotalPositions) revert MaxTotalPositionsReached();
5   + // Prevent cross-DEX tokenId collision from overwriting an existing position
6   + if (positions[tokenId].owner != address(0)) revert PositionAlreadyExists();
7   positions[tokenId] = UserPosition({...});
```

Team Response

Fixed: Fixed on both Arbitrum deployments (guard live) + in codebase for future; both Base deployments risk-accepted (won't-fix — live-TVL upgrade risk > near-zero collision)

[L-02] calculateCenteredRange Mints One-Sided, Off-Center Positions When rangeWidthBps <= tickSpacing

Fixed

`TickMath.calculateCenteredRange` is supposed to wrap a symmetric band around the pool's current tick. Whenever `rangeWidthBps <= tickSpacing`, it doesn't. Two small integer-math bugs compound and the user ends up with a one-sided strip that isn't centered on anything the market cares about.

Here's the first bug. When the stored width equals a single `tickSpacing`, `halfWidth` quietly collapses to zero:

```
1 int24 halfWidth = (roundedWidth / 2 / tickSpacing) * tickSpacing;
2 // rangeWidthBps == tickSpacing → (tickSpacing / 2 / tickSpacing) * tickSpacing = 0
```

tickSpacing / 2 / tickSpacing is 0.5 in real math, but Solidity floors it to 0, so halfWidth becomes zero. The fallback at line 57 then kicks in: tickUpper = centerTick + tickSpacing, tickLower = centerTick. The range comes out as [centerTick, centerTick + tickSpacing], everything above the center, nothing below. This hits every fee tier the protocol supports.

Widths below tickSpacing get funneled into the same bug by _roundWidth:

```
1 int24 roundedWidth = (int256(uint256(rangeWidthBps)) / tickSpacing) * tickSpacing;
2 if (roundedWidth < tickSpacing) roundedWidth = tickSpacing; // fallback
```

So rangeWidthBps = 50 on a tickSpacing = 60 pool is silently promoted to 60, which then collapses halfWidth to zero exactly as above. The default minRangeWidthBps = 50 is inside the unsafe zone on the 0.30% (tickSpacing=60) and 1% (tickSpacing=200) pools, which means users don't even need to opt in to a weird preset, just accepting the default produces the bug.

The second bug stacks on top. centerTick isn't the real current tick, roundToNearestSpacing snaps it via integer truncation toward zero, so negative ticks round up and positive ticks round down. Unless the current tick is already a multiple of tickSpacing, the center is offset from the market by up to tickSpacing - 1. Harmless on its own, but combined with bug one the one-sided chunk gets glued to the wrong side of an already-offset center, and the entire range can end up sitting above or below where the market actually is.

There are two independent entry points that feed bad widths into this math:

1. Deposit / single-sided deposit. _createPosition accepts any rangeWidthBps that passes the global [minRangeWidthBps, maxRangeWidthBps] check. Since minRangeWidthBps default is 50 and pool tickSpacing isn't consulted, widths in [minRangeWidthBps, tickSpacing] pass on the 0.30% and 1% pools and mint one-sided from day one.
2. updateParameters on a live position. Any position owner can change their own rangeWidthBps at any time:

```
1 function updateParameters(uint256 tokenId, ..., uint24 newRangeWidthBps, ...) external
   nonReentrant {
2     UserPosition storage pos = positions[tokenId];
3     if (pos.owner != msg.sender) revert NotPositionOwner();
4     _validateParameters(newRangeWidthBps, newRebalanceDelay); // only global bounds, no
       tickSpacing
5     pos.rangeWidthBps = newRangeWidthBps;
6     //...
7 }
```

_validateParameters only checks the same global band. The new width is stored, and the next keeper rebalance or manualSnuggle feeds it straight into calculateCenteredRange. A position that was originally minted with a safe width can be flipped into the one-sided mint simply by the owner tweaking their own settings, no redeposit required, no cross-check against the pool's tickSpacing.

The same truncation quietly collapses distinct presets into the same position on wider widths too. On a 0.30% pool, both the 1.2% and 1.8% width presets compute halfWidth = 60 and mint an identical ±60 tick range. Users pick "wider for safety" and get the same thing as "tighter for fees". The rounding silently throws their choice away.

Recommendation:

Fix the math so sub-tickSpacing widths can't silently collapse, and validate against the destination pool's tickSpacing on every write path, not just the global bounds:

```
1 // TickMath.calculateCenteredRange
2 int24 halfWidth = ((roundedWidth / 2 + tickSpacing / 2) / tickSpacing) * tickSpacing;
```



```
3 require(halfWidth > 0, "Range too narrow for tick spacing");
4
5 // Wherever rangeWidthBps is accepted (deposit, depositSingleSided, updateParameters)
6 require(newRangeWidthBps >= 2 * poolConfig.tickSpacing, "Range below 2x tickSpacing");
```

Also fix `roundToNearestSpacing` to pick the genuinely closest multiple of `tickSpacing` instead of whichever is closer to zero, so the center isn't biased on negative ticks. For defense in depth, raise `minRangeWidthBps` to at least $2 * \text{maxTickSpacing}$ across all approved pools so the trigger window can't be reached through the global bounds alone. Finally, sweep existing affected positions by matching `tickUpper - tickLower == tickSpacing` against the vault's NFT set and running them through the keeper once the math fix is deployed.

Team Response

Fixed: Library math fix + $2 \times \text{tickSpacing}$ floor, in codebase for future deployments

[L-03] Retroactive performance fee application charges existing positions at new rate on already-accrued earnings

Acknowledged

SnuggleVaultUpgradeable stores a single global `performanceFeeBps` variable that is applied to all fee and reward distributions. When a user deposits and creates a position, no per-position fee rate is recorded in the `UserPosition` struct. Instead, the current global rate is read at the time of every harvest, withdrawal, rebalance, or reward claim via `StakingManager.distributePerformanceFees()`. When the vault owner changes `performanceFeeBps` via `AdminSatellite.setPerformanceFee()`, the new rate applies immediately to all existing positions' next fee distribution, including earnings that accrued entirely under the previous rate. A user who deposited when the fee was 15% and accrued trading fees over weeks will have those already-earned fees charged at the new rate (up to the 50% cap) when they next harvest or withdraw.

```
1 // StakingManager.sol:378 - reads current global rate, not deposit-time rate
2 uint256 feeBps = v.performanceFeeBps();
3 uint256 fee0 = (earned0 * feeBps) / BPS_DENOMINATOR;
4 uint256 fee1 = (earned1 * feeBps) / BPS_DENOMINATOR;
```

The setter in `AdminSatellite` enforces a 6-hour cooldown (`FEE_CHANGE_COOLDOWN`) and a maximum 500bps change per update (`MAX_FEE_CHANGE_BPS`), but does not settle existing positions first:

```
1 // SnuggleVaultAdminSatellite.sol:228-240
2 function setPerformanceFee(uint256 newFeeBps) external onlyVaultOwner {
3     if (newFeeBps > MAX_PROTOCOL_FEE_BPS) revert FeeTooHigh();
4     if (lastFeeChangeTime > 0 && block.timestamp < lastFeeChangeTime + FEE_CHANGE_COOLDOWN) {
5         revert FeeChangeCooldown();
6     }
7     uint256 current = vault.performanceFeeBps();
8     uint256 delta = newFeeBps > current ? newFeeBps - current : current - newFeeBps;
9     if (delta > MAX_FEE_CHANGE_BPS) revert FeeChangeTooLarge();
10    lastFeeChangeTime = block.timestamp;
11    vault.setPerformanceFee(newFeeBps);
12 }
```

The `UserPosition` struct stores cumulative fee counters (`cumulativeFees0`, `cumulativeFees1`, `cumulativeRewards`) for APR tracking but no snapshot of the fee rate at deposit time. There is no mechanism for a user to lock in their fee rate or force settlement of accrued earnings before a fee change takes effect.

Record the performanceFeeBps at deposit time in the UserPosition struct and use that rate for the lifetime of the position. Alternatively, emit a fee change event with a delay (e.g. 7 days) so users can harvest their accrued earnings at the old rate before the new rate takes effect.

Team Response

Acknowledged: Won't-fix — global-rate design; fee never changed (pre-announced if ever); no principal risk

[L-04] renounceOwnership not blocked in SnuggleVaultUpgradeable**Acknowledged**

SnuggleVaultUpgradeable inherits from Ownable2StepUpgradeable without overriding renounceOwnership(). Calling renounceOwnership() sets the owner to address(0) permanently, disabling every onlyOwner function in the vault: pause(), unpause(), setAdminSatellite(), and proxy upgrade authorization. This is an irreversible action with no recovery path.

Implement the same function that KeepersHelper.sol has

```
1 // KeepersHelper.sol:L84-85
2 function renounceOwnership() public pure override {
3     revert("renounceOwnership disabled");
4 }
```

Team Response

Acknowledged: Won't-fix — intentional capability

[L-05] Missing reward adapter validation in updatePoolRewardAdapter**Fixed**

addPoolDirect validates that a reward adapter is compatible with the target pool via IRewardAdapter(rewardAdapter).validatePool(pool), catching configuration errors at registration time. However, updatePoolRewardAdapter skips this validation entirely. This means the adapter may reference a non-existent gauge or an unregistered MasterChef, causing all subsequent staking operations on that pool to silently fail or revert.

```
1 function updatePoolRewardAdapter(bytes32 poolId, address newRewardAdapter) external
  onlyVaultOwner {
2     // NOTE here there must be the same check as above to validate reward adapter can
      actually stake in this pool
3     vault.updatePoolRewardAdapter(poolId, newRewardAdapter);
4 }
```

An incompatible reward adapter silently breaks staking for all positions in the affected pool. Users cannot earn rewards until the owner notices and corrects the adapter. Uncomment and implement the validation check, mirroring the logic in addPoolDirect. The pool address needs to be retrieved from the vault's pool config:

```
1 function updatePoolRewardAdapter(bytes32 poolId, address newRewardAdapter) external
  onlyVaultOwner {
2     if (newRewardAdapter != address(0)) {
3         address pool = adapter.getPool(token0, token1, fee);
4         try IRewardAdapter(newRewardAdapter).validatePool(pool) {}
```

```
5         catch { revert RewardAdapterValidationFailed(); }
6     }
7     vault.updatePoolRewardAdapter(poolId, newRewardAdapter);
8 }
```

Team Response

Fixed: Validation fix (satellite) in codebase for future; on live deployments covered by the M-05 operational invariant (adapters never rotated on live pools)

[L-06] Adding a pool with a new token breaks fee harvesting until the admin re-registers the helper

Acknowledged

FeeTransferHelper moves fees out of the vault via `safeTransferFrom(vault, recipient, amount)`, so it needs an allowance on every token it will ever transfer. Allowances are only granted as a side effect of `addPoolDirect`:

```
1 // snuggle-base/SnuggleVaultUpgradeable.sol:L411-L412
2 IERC20(token0).approve(_positionAdapter, type(uint256).max);
3 IERC20(token1).approve(_positionAdapter, type(uint256).max);
```

The helper is registered once at deploy time with a single reference token pair, so it only has allowances on those two tokens. When the admin later calls `satellite.addPool` with a real adapter and a new token, the vault approves the real adapter but never the helper. The new pool looks healthy, users deposit, fees accrue — and then the first harvest reverts with zero allowance on the new token. Impact. All fee paths revert for the affected pool: `distributePerformanceFees` (StakingManager.sol:L358), `processFeesForCompounding` (L299), and `_harvestAllFees` (L423). The referral fallback at L450-L452 calls the same broken helper and also reverts. User principal is safe (`withdraw()` uses `adapter.collect()` directly at L584), but accrued fees sit stranded in the vault until the admin intervenes. Nothing warns at registration time — the misconfiguration only surfaces on the first harvest.

Recovery exists but is undocumented. The admin can call `satellite.addPool` a second time with the helper as `positionAdapter` and the new pair; `addPoolDirect` then grants the helper max allowance on those tokens. The resulting `PoolConfig` is cosmetically junk but inert (every mint on the helper reverts `NotImplemented`). The deployment comment at `FeeTransferHelper.sol:L38-L44` does not mention this step. Recommendation. Preferred fix — in `addPoolDirect`, also approve the helper whenever a new pair is added:

```
1 IERC20(token0).approve(_positionAdapter, type(uint256).max);
2 IERC20(token1).approve(_positionAdapter, type(uint256).max);
3 + if (address(stakingManager) != address(0)) {
4 +   address helperAddr = address(stakingManager.feeTransferHelper());
5 +   if (helperAddr != address(0)) {
6 +     IERC20(token0).approve(helperAddr, type(uint256).max);
7 +     IERC20(token1).approve(helperAddr, type(uint256).max);
8 +   }
9 + }
```

Team Response

Acknowledged: Won't-fix — operational; already standard deploy step, formalized in add-pool runbook

[L-07] Tick boundary clamping can produce inverted ranges at extreme ticks**Fixed**

Every range builder in TickMath ends with the same one-sided clamp: tickLower gets raised to MIN_TICK, tickUpper gets lowered to MAX_TICK, but the two are never compared afterward. When currentTick sits within roundedWidth of a boundary, the upstream math can push the raw bounds past the limit while only one side gets clipped — the result is an interval where tickLower > tickUpper.

```
1 // TickMath.sol:122-146 — _buildDirectionalRange
2 if (isToken0) {
3     tickLower = roundUpToSpacing(currentTick, tickSpacing);
4     if (tickLower <= currentTick) tickLower += tickSpacing;
5     tickUpper = tickLower + roundedWidth;
6 } else {
7     tickUpper = roundDownToSpacing(currentTick, tickSpacing);
8     if (tickUpper >= currentTick) tickUpper -= tickSpacing;
9     tickLower = tickUpper - roundedWidth;
10 }
11
12 if (tickLower < MIN_TICK) tickLower = MIN_TICK; // only clamps low side
13 if (tickUpper > MAX_TICK) tickUpper = MAX_TICK; // only clamps high side
```

Concrete example in _buildDirectionalRange with isToken0 = true, currentTick = 887270, tickSpacing = 60, roundedWidth = 600:

```
1 roundUpToSpacing(887270, 60) → 14788 * 60 = 887280
2 tickLower (887280) <= currentTick (887270) is false → no adjustment
3 tickUpper = 887280 + 600 = 887880
4 Clamps: tickLower is above MAX_TICK but not checked; tickUpper is clipped to 887272
5 Returned: tickLower = 887280, tickUpper = 887272 — inverted
```

The same pattern hits calculateCenteredRange at the minimum-width preset near MAX_TICK, and the symmetric case hits token1-dominant mints near MIN_TICK. Uniswap's pool.mint rejects the bad range via require(tickLower < tickUpper, 'TLU'), so the failure mode is a deterministic deposit/rebalance revert — no funds move, no state corrupts. In executeRebalance, the mint reverts atomically with the preceding decreaseLiquidity, so positions are not left in a stranded state.

Reachability on Snuggle's current pools is essentially zero. MAX_TICK = 887272 encodes a price ratio near 1.46e38, which no realistic token pair reaches (WETH/USDC tops out near ±200k, extreme meme pairs near ±400k). The risk only materializes if the protocol onboards pairs with extreme decimal mismatches that naturally trade near the tick limits.

Add a post-clamp ordering assertion to every range builder so the failure surfaces with a protocol-specific error at the library layer instead of bubbling up as an opaque Uniswap revert:

```
1 if (tickLower < MIN_TICK) tickLower = MIN_TICK;
2 if (tickUpper > MAX_TICK) tickUpper = MAX_TICK;
3 if (tickLower >= tickUpper) revert InvalidRangeAtBoundary();
```

Alternatives: a dedicated approveTokenForHelper(token) admin function on the satellite, or at minimum a documented step 6 in the deployment comment warning operators that every new token pair requires a second addPool call with the helper as adapter.

Team Response

Fixed: Planned — library ordering assert

[I-01] clearReferrer leaves stale data in ReferralTracker when reassigning to a different referrer**Fixed**

The vault's `clearReferrer(user)` only deletes `referrer[user]` on the vault side. The separate `ReferralTracker` contract keeps its own `userReferralIndex[user]` mapping, which is never touched and has no admin function to clear it.

If the user is later reassigned to a different referrer, the vault accepts the change. But on the next referral fee distribution, `StakingManager.safeTransferReferral` sends the fee to the new referrer and then calls `ReferralTracker.recordReferral(user)` to register the relationship. This call reverts with `AlreadyRecorded` because the stale index still points to the old referrer. The revert is swallowed by the surrounding try-catch, leaving the tracker permanently inconsistent:

```
1 vault.referrer[user] = newReferrer
2 ReferralTracker.referrals[oldReferrer] = [user] // stale
3 ReferralTracker.referrals[newReferrer] = []      // missing
```

Payouts and earnings accounting still work correctly because both read from the vault, not from `userReferralIndex`. The broken pieces are the on-chain relationship queries: `getReferrals`, `getReferralCount`, and `isReferralRecorded` return stale or misleading data for the affected user. Reassigning to the same original referrer is not affected.

Recommendation: Add a `clearReferral(address user)` function to `ReferralTracker`, restricted to the vault owner, and invoke it from `clearReferrer` so both sides are reset atomically. Alternatively, drop the `AlreadyRecorded` revert in `recordReferral` and let it re-sync whenever the vault's referrer has changed.

Team Response

Fixed: Planned — tracker clear fn